



Shortest paths

Martedì 19 maggio 2026

Grafo orientato pesato

- + Un **Grafo orientato pesato** (edge-weighted digraph) è un grafo orientato in cui ogni arco ha un peso (o costo) associato.

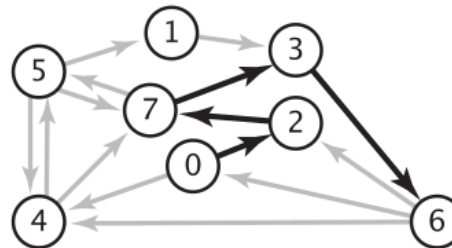
applicazione	vertice	arco
mappe stradali	incroci	strade (a senso unico)
scheduling	task	relazioni di precedenza
routing (telecom)	nodi	connessioni

Cammini minimi

- + Dato un grafo orientato pesato G , trova il percorso più corto (o più leggero) dal vertice s al vertice t .

edge-weighted digraph

4→5	0.35
5→4	0.35
4→7	0.37
5→7	0.28
7→5	0.28
5→1	0.32
0→4	0.38
0→2	0.26
7→3	0.39
1→3	0.29
2→7	0.34
6→2	0.40
3→6	0.52
6→0	0.58
6→4	0.93



shortest path from 0 to 6

0→2	0.26
2→7	0.34
7→3	0.39
3→6	0.52

Variazione sui cammini minimi

- + Vertici:
 - + Single source: da s a ogni altro vertice;
 - + Single-sink: da un qualsiasi vertice al vertice t ;
 - + Source-sink: da un vertice s ad un vertice t
 - + All-pairs: tra ogni coppia di vertici del grafo.
- + Pesi:
 - + Pesi non negativi;
 - + Pesi euclidei (distanze);
 - + Pesi arbitrari (anche negativi).
- + Cicli:
 - + Nessun ciclo orientato;
 - + Nessun ciclo negativo.
- + Semplificazione: supponiamo che il cammino minimo esista, anche se non è necessariamente unico.

Una classe per gli archi orientati pesati

```
public class DirectedEdge  
  
    DirectedEdge(int v, int w, double weight)  
    int from()  
    int to()  
    double weight()  
    String toString()
```

Elaborazione di un edge:

```
+ int v = e.from(), w = e.to(v);
```

Implementazione

```
public class DirectedEdge {  
    private final int v;  
    private final int w;  
    private final double weight;  
  
    public DirectedEdge(int v, int w, double weight) {  
        this.v = v;  
        this.w = w;  
        this.weight = weight;  
    }  
  
    public int from() {  
        return v;  
    }  
  
    public int to() {  
        return w;  
    }  
  
    public double weight() {  
        return weight;  
    }  
}
```

Edge-weighted digraph: la classe

```
public class EdgeWeightedDigraph
```

```
    EdgeWeightedDigraph(int V)
```

```
    EdgeWeightedDigraph(In in)
```

```
    void addEdge(DirectedEdge e)
```

```
    Iterable<DirectedEdge> adj(int v)
```

```
    Iterable<DirectedEdge> edges()
```

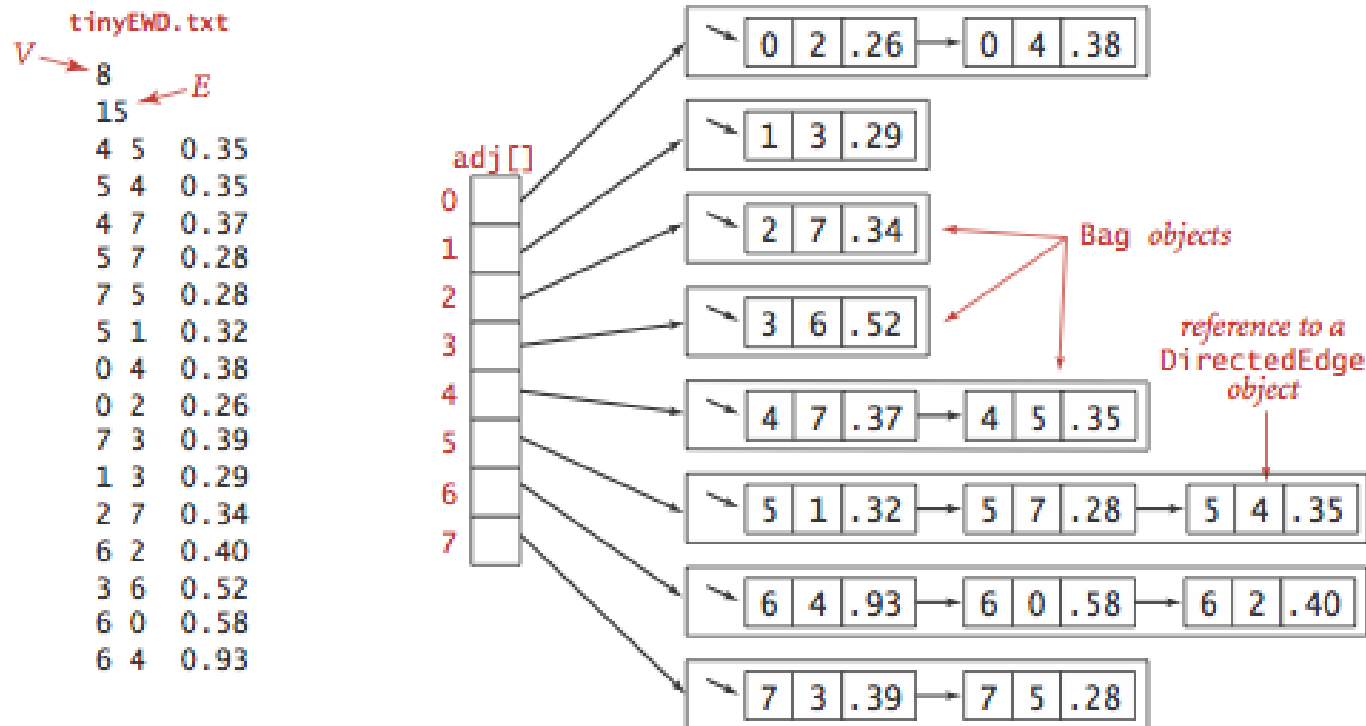
```
    int V()
```

```
    int E()
```

```
    String toString()
```

- + Self-loop e archi paralleli sono consentiti, anche se solo uno degli archi paralleli può rientrare in un shortest path, e un self-loop può rientrare solo se a peso nullo.

Grafi orientati pesati: rappresentazione



Edge-weighted digraph representation

Grafi orientati pesati: implementazione

```
public class EdgeWeightedDigraph {
    private final int V;
    private int E;
    private Bag<DirectedEdge>[] adj;
    private int[] indegree;           // indegree[v] = indegree of vertex v

    public EdgeWeightedDigraph(int V) {
        this.V = V; this.E = 0;
        this.indegree = new int[V];
        adj = (Bag<DirectedEdge>[]) new Bag[V];
        for (int v = 0; v < V; v++)
            adj[v] = new Bag<DirectedEdge>();
    }

    public void addEdge(DirectedEdge e) {
        int v = e.from(); int w = e.to();
        adj[v].add(e);
        indegree[w]++;
        E++;
    }

    public Iterable<DirectedEdge> adj(int v) {
        return adj[v];
    }
}
```

Single source shortest paths: classe e client

```
public class SP
```

```
    SP(EdgeWeightedDigraph G, int s)
```

```
    double distTo(int v)
```

```
    Iterable<DirectedEdge> pathTo(int v)
```

```
    boolean hasPathTo(int v)
```

```
public static void main(String[] args) {
```

```
    In in = new In(args[0]);
```

```
    EdgeWeightedDigraph G = new EdgeWeightedDigraph(in);
```

```
    int s = Integer.parseInt(args[1]);
```

```
    SP sp = new SP(G, s);
```

```
    for (int t = 0; t < G.V(); t++) {
```

```
        StdOut.printf("%d to %d (%.2f)\n", s, t, sp.distTo(t));
```

```
        for (DirectedEdge e : sp.pathTo(t))
```

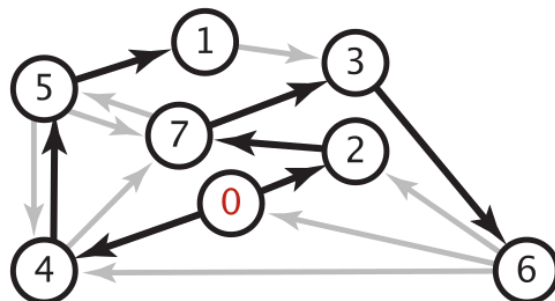
```
            StdOut.print(e + " ");
```

```
        StdOut.println();
```

```
}
```

Albero dei cammini minimi

- + La soluzione del problema single-source SP è un albero chiamato shortest-paths tree (SPT).
- + Tale albero esiste: se ci sono due cammini dello stesso peso da s a un nodo, si scarta l'ultimo arco di uno dei due;
- + Lo SPT si rappresenta con due array:
 - + $\text{distTo}[v]$ contiene la lunghezza (peso) dello SP da s a v ;
 - + $\text{edgeTo}[v]$ è l'ultimo arco dello SP da s a v .



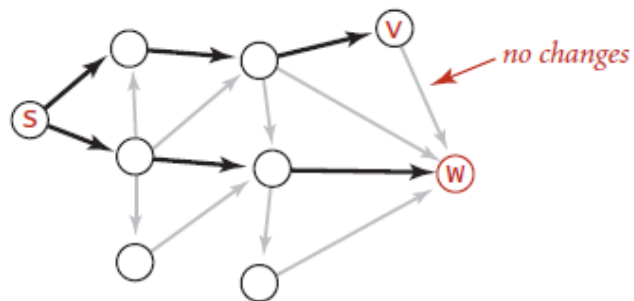
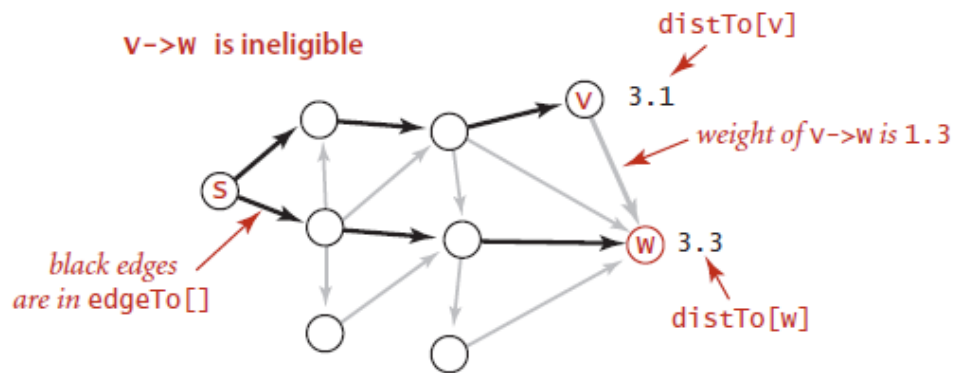
	edgeTo[]	distTo[]
0	null	0
1	5->1 0.32	1.05
2	0->2 0.26	0.26
3	7->3 0.37	0.97
4	0->4 0.38	0.38
5	4->5 0.35	0.73
6	3->6 0.52	1.49
7	2->7 0.34	0.60

Rilassamento di un arco $e=v \rightarrow w$

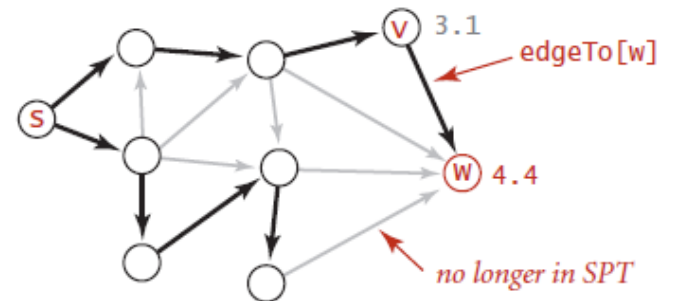
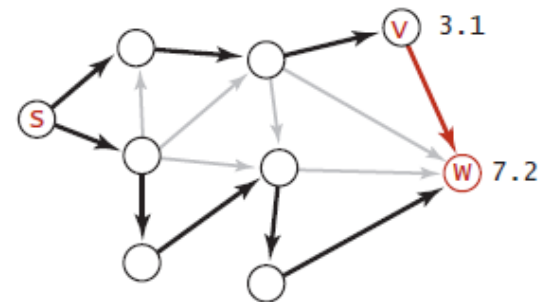
- + $\text{distTo}[v]$ contenga la lunghezza del più corto percorso scoperto finora da s a v ;
- + $\text{distTo}[w]$ contenga la lunghezza del più corto percorso scoperto finora da s a w , e sia $\text{edgeTo}[w]$ l'ultimo arco di tale percorso;
- + se l'arco e consente di produrre un cammino più breve da s a w (via v) allora si aggiornino $\text{distTo}[w]$ e $\text{edgeTo}[w]$.

```
private void relax(DirectedEdge e) {  
    int v = e.from(), w = e.to();  
    if (distTo[w] > distTo[v] + e.weight()) {  
        distTo[w] = distTo[v] + e.weight();  
        edgeTo[w] = e;  
    }  
}
```

Due casi



v → w is eligible



Edge relaxation (two cases)

Condizione di ottimalità per SP

- + Teorema: $\text{distTo}[]$ contiene le distanze dei SP da s se solo se
 1. $\text{distTo}[s] = 0$;
 2. Per ogni vertice v , $\text{distTo}[v]$ è la lunghezza di un cammino da s a v ;
 3. Per ogni arco $e = v \rightarrow w$, $\text{distTo}[w] \leq \text{distTo}[v] + e.\text{weight}()$.

- + Dimostrazione (Condizione necessaria)

Se esiste $e = v \rightarrow w$ tale che $\text{distTo}[w] > \text{distTo}[v] + e.\text{weight}()$, allora esiste un cammino da s a w di lunghezza minore di $\text{distTo}[w]$.

Condizione di ottimalità per SP

- + Teorema: $\text{distTo}[]$ contiene le distanze dei SP da s se solo se
 - + $\text{distTo}[s] = 0$;
 - + Per ogni vertice v , $\text{distTo}[v]$ è la lunghezza di un cammino da s a v ;
 - + Per ogni arco $e = v \rightarrow w$, $\text{distTo}[w] \leq \text{distTo}[v] + e.\text{weight}()$.

- + Dimostrazione (Condizione sufficiente)

Dato uno SP da s a w , supponiamo che Opt sia il suo peso.

Per tutti i suoi vertici contigui vale

$$\text{distTo}[v_k] \leq \text{distTo}[v_{k-1}] + e_k.\text{weight}(),$$

$$\text{distTo}[v_{k-1}] \leq \text{distTo}[v_{k-2}] + e_{k-1}.\text{weight}(),$$

.....

e sommando le disuguaglianze per ogni k si ottiene:

$$\text{distTo}[w] \leq e_1.\text{weight}() + \dots + e_k.\text{weight}() = \text{Opt};$$

Ma $\text{distTo}[w]$ è la lunghezza di un cammino (non più corto di uno SP), e quindi:

$$\text{Opt} \leq \text{distTo}[w] \leq \text{Opt}.$$

Algoritmo generico per il calcolo del SPT

- + Il seguente algoritmo calcola uno SPT (se esiste) da s :
 - + Inizializza $\text{distTo}[s] = 0$ e $\text{distTo}[v] = \infty$ per ogni altro vertice;
 - + Ripeti il rilassamento di un arco fino a quando sono soddisfatte le condizioni di ottimalità.

- + Dimostrazione di correttezza:

Se un rilassamento ha successo allora esso diminuisce $\text{distTo}[v]$ per qualche v ;

le possibili diminuzioni di $\text{distTo}[v]$ sono in numero finito;

quando nessun rilassamento è più possibile si applicano le condizioni di ottimalità.

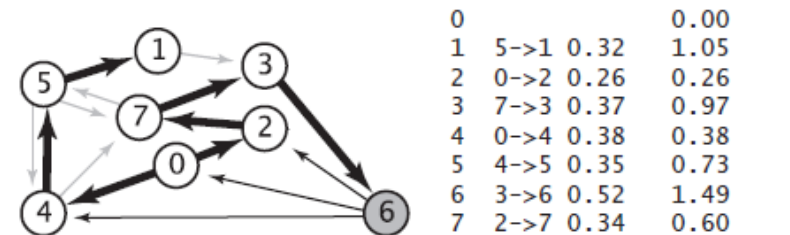
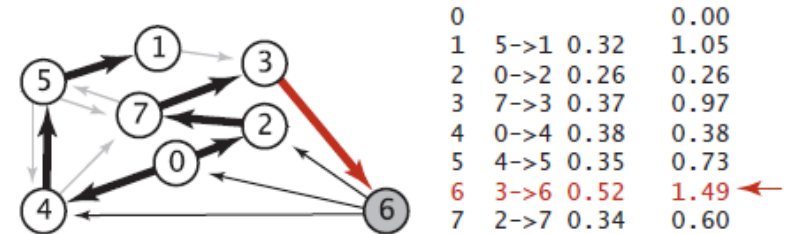
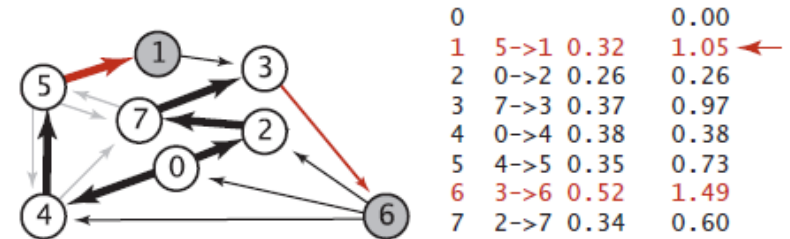
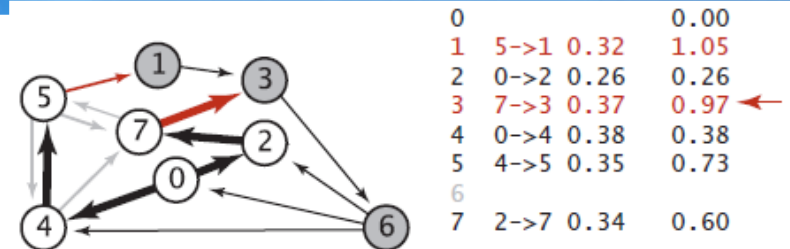
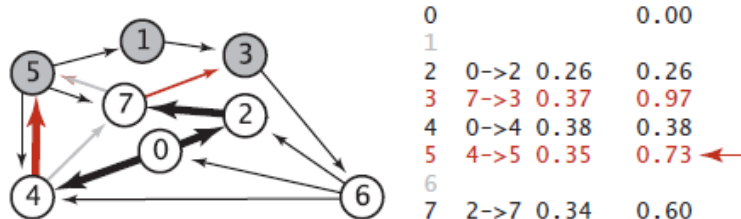
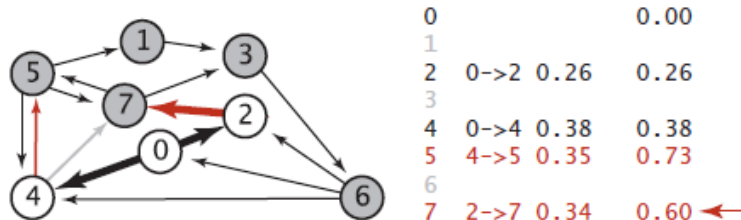
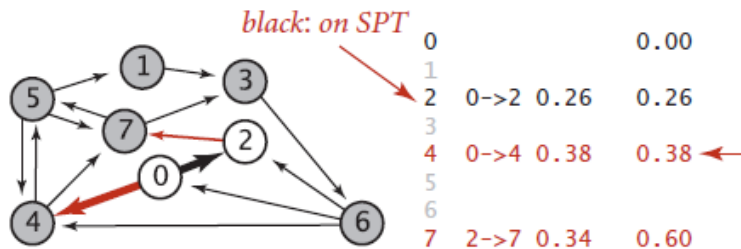
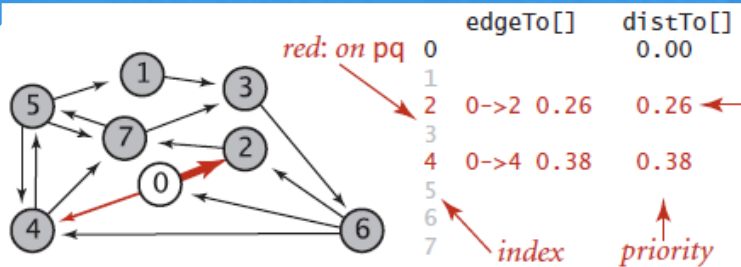
Come scegliere l'arco da rilassare

- + Come scegliere il prossimo arco da rilassare:
 - + Algoritmo di Dijkstra (pesi non negativi);
 - + Topological sort (grafo privo di cicli orientati);
 - + Algoritmo di Bellman-Ford (grafo privo di cicli negativi).
- + Per dimostrare che un algoritmo calcola gli SP è sufficiente dimostrare che rilassa archi fino a non trovarne più di rilassabili.

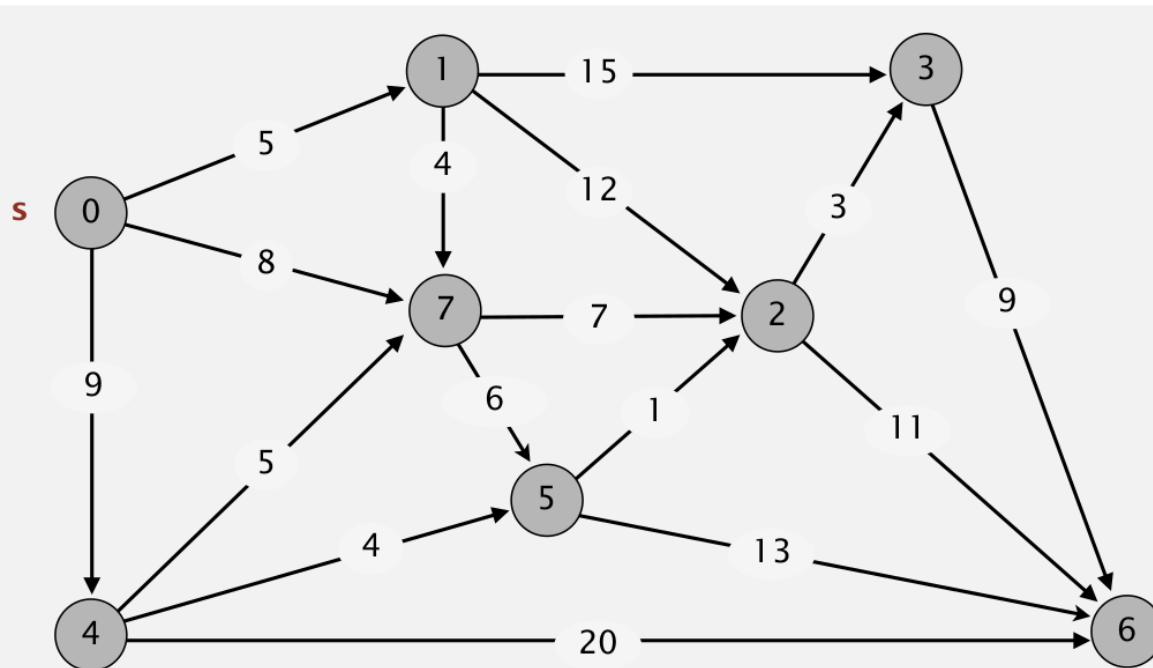
Algoritmo di Dijkstra

1. Considera i vertici in ordine crescente di peso (distanza) dalla sorgente, cioè i vertici che non appartengono all'albero e che hanno il più basso valore di $\text{distTo}[]$ (inizialmente sono tutti a ∞ eccetto s);
2. Aggiungi il vertice più vicino a s e rilassa tutti gli archi che partono da quel vertice.
3. Ripeti (2) fino a che tutti i vertici sono nell'albero o non esistono vertici esterni all'albero con valore finito di $\text{distTo}[]$.

Algoritmo di Dijkstra: esempio



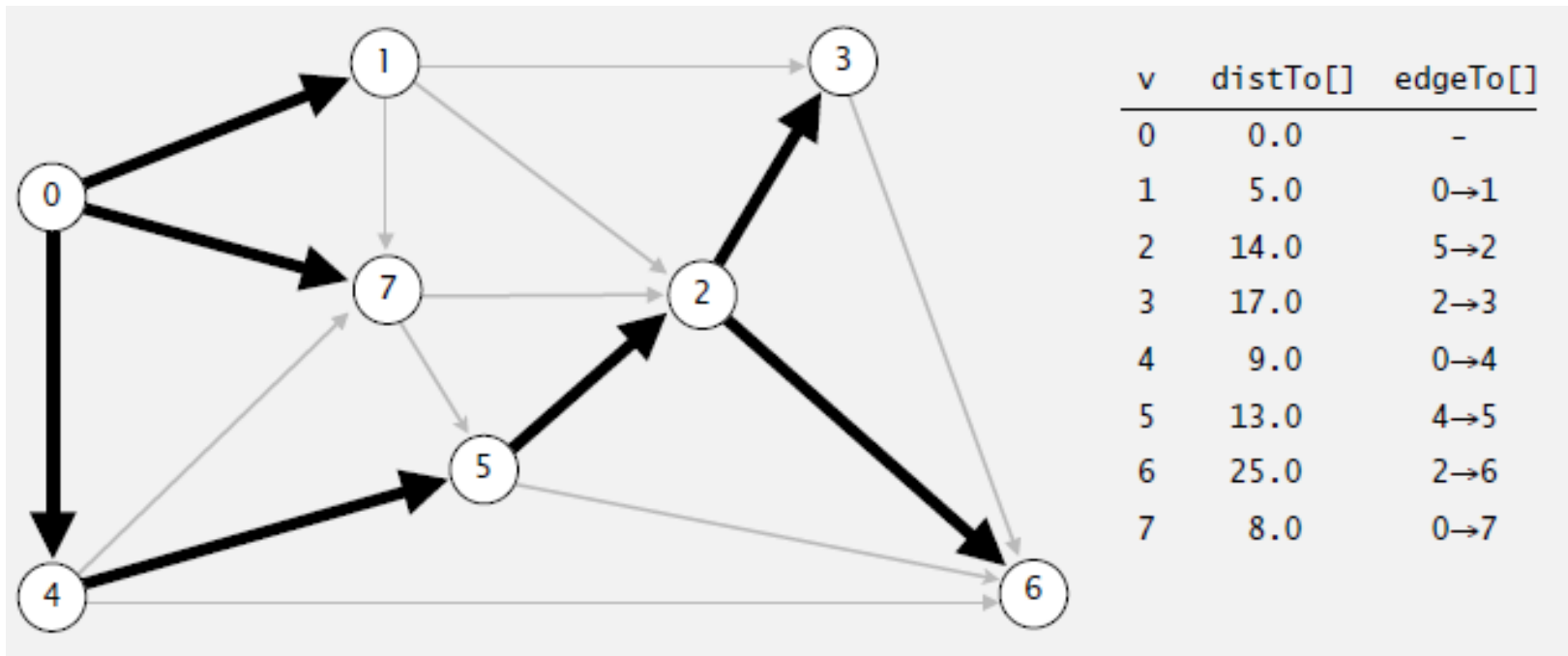
Algoritmo di Dijkstra: esercizio



an edge-weighted digraph

0→1	5.0
0→4	9.0
0→7	8.0
1→2	12.0
1→3	15.0
1→7	4.0
2→3	3.0
2→6	11.0
3→6	9.0
4→5	4.0
4→6	20.0
4→7	5.0
5→2	1.0
5→6	13.0
7→5	6.0
7→2	7.0

Algoritmo di Dijkstra: SPT



Algoritmo di Dijkstra: correttezza

- + Teorema: L'algoritmo di Dijkstra trova uno SPT in un grafo orientato pesato con pesi non negativi.
- + Dimostrazione: Ogni arco $e = v \rightarrow w$ viene rilassato una sola volta (da v), lasciando $\text{distTo}[w] \leq \text{distTo}[v] + e.\text{weight}()$. La disuguaglianza vale fino alla fine perché $\text{distTo}[w]$ non può aumentare e $\text{distTo}[v]$ non cambierà (visto che i pesi sono non negativi). Alla fine, vale la condizione di ottimalità.

Algoritmo di Dijkstra: implementazione

```
public class DijkstraSP {
    private double[] distTo;
    private DirectedEdge[] edgeTo;
    private IndexMinPQ<Double> pq;

    public DijkstraSP(EdgeWeightedDigraph G, int s) {
        distTo = new double[G.V()];
        edgeTo = new DirectedEdge[G.V()];
        for (int v = 0; v < G.V(); v++)
            distTo[v] = Double.POSITIVE_INFINITY;
        distTo[s] = 0.0;
        pq = new IndexMinPQ<Double>(G.V());
        pq.insert(s, distTo[s]);
        while (!pq.isEmpty()) {
            int v = pq.delMin();
            for (DirectedEdge e : G.adj(v))
                relax(e);
        }
    }
    ...
}
```

Algoritmo di Dijkstra: implementazione

```
...
private void relax(DirectedEdge e) {
    int v = e.from(), w = e.to();
    if (distTo[w] > distTo[v] + e.weight()) {
        distTo[w] = distTo[v] + e.weight();
        edgeTo[w] = e;
        if (pq.contains(w)) pq.decreaseKey(w, distTo[w]);
        else pq.insert(w, distTo[w]);
    }
}

public double distTo(int v) {
    return distTo[v];
}

public boolean hasPathTo(int v) {
    return distTo[v] < Double.POSITIVE_INFINITY;
}

public Iterable<DirectedEdge> pathTo(int v) {
    if (!hasPathTo(v)) return null;
    Stack<DirectedEdge> path = new Stack<DirectedEdge>();
    for (DirectedEdge e = edgeTo[v]; e != null; e = edgeTo[e.from()]) {
        path.push(e);
    }
    return path;
}
```


Algoritmo di Dijkstra: complessità

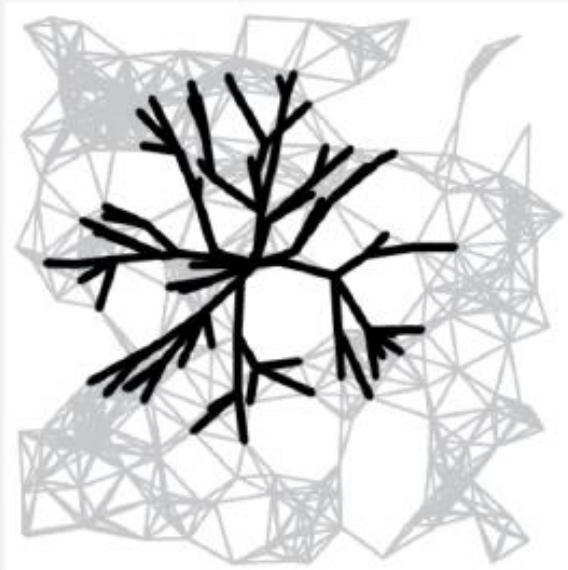
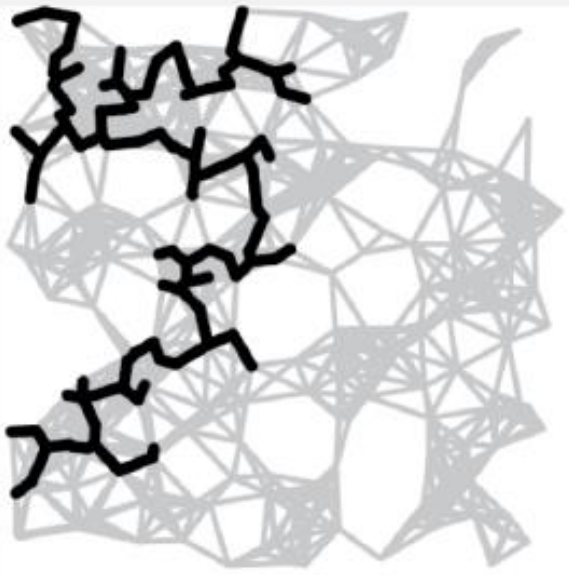
- + La complessità in tempo dipende dall'implementazione della PQ, essendoci
V op. insert, V op. delete-min, E op. decrease-key.

PQ	insert	delete-min	decrease-key	total
binary heap	$\log V$	$\log V$	$\log V$	$E \log V$

- + La complessità in spazio è proporzionale a V.

Dijkstra vs Prim

Entrambi costruiscono un albero un arco alla volta:
aggiungi il vertice più vicino all'albero (Prim) o il più vicino
alla sorgente (attraverso un cammino orientato - Dijkstra);



Algoritmo di Dijkstra: altri problemi affrontabili

- + SP da una sorgente in grafi non orientati: Ogni arco viene tradotto in due archi orientati con lo stesso peso.
- + Source-sink SP: Arresta l'algoritmo non appena il sink viene estratto dalla priority queue.
- + All-pairs SP: Esegui l'algoritmo per ogni vertice del grafo nel ruolo di sorgente.

```
public class DijkstraAllPairsSP
{
    private DijkstraSP[] all;
    DijkstraAllPairsSP(EdgeWeightedDigraph G)
    {
        all = new DijkstraSP[G.V()]
        for (int v = 0; v < G.V(); v++)
            all[v] = new DijkstraSP(G, v);
    }
    Iterable<Edge> path(int s, int t)
    { return all[s].pathTo(t); }
    double dist(int s, int t)
    { return all[s].distTo(t); }
}
```

Grafi orientati pesati aciclici

Consideriamo il problema di trovare i cammini minimi nel caso di grafi aciclici. In questo caso è possibile trovare un algoritmo più semplice e veloce di Dijkstra, che

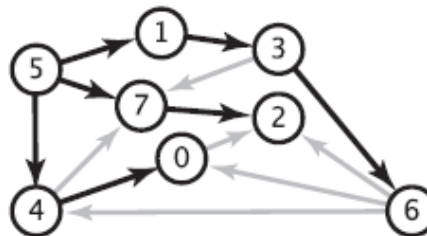
- Risolve il single –source problem in tempo lineare;
- Gestisce archi con pesi negativi
- Risolve problemi correlati, come trovare il cammino più lungo.

Topological sort per SPT

1. Si esegue l'ordinamento topologico;
2. Seguendo l'ordinamento si prende un vertice e si rilassano tutti gli archi che partono da quel vertice.

tinyEWDAG.txt

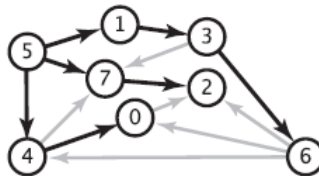
V → 8
13 ← E
5 4 0.35
4 7 0.37
5 7 0.28
5 1 0.32
4 0 0.38
0 2 0.26
3 7 0.39
1 3 0.29
7 2 0.34
6 2 0.40
3 6 0.52
6 0 0.58
6 4 0.93



Topological sort per SPT: esempio

tinyEWDAG.txt

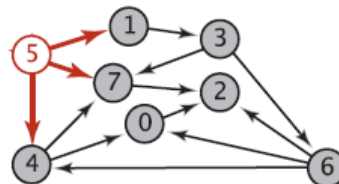
V → 8
E
13
5 4 0.35
4 7 0.37
5 7 0.28
5 1 0.32
4 0 0.38
0 2 0.26
3 7 0.39
1 3 0.29
7 2 0.34
6 2 0.40
3 6 0.52
6 0 0.58
6 4 0.93



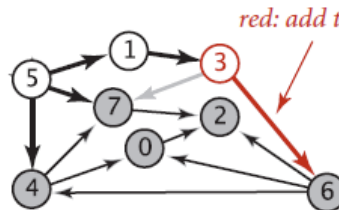
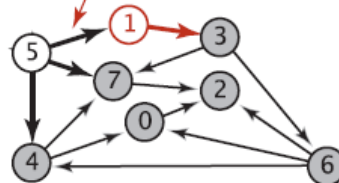
An acyclic edge-weighted digraph with an SPT

topological sort

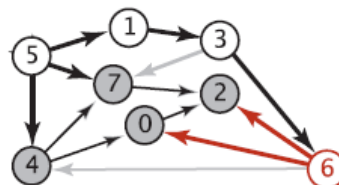
5 1 3 6 4 7 0 2



thick black: on tree



red: add to tree



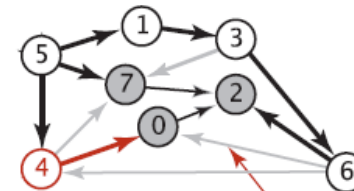
edgeTo[]

0
1 5->1
2
3
4 5->4
5
6
7 5->7

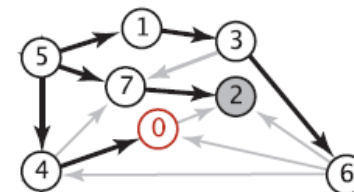
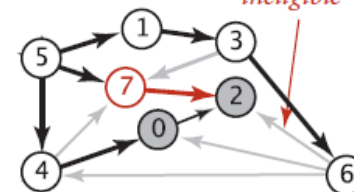
0
1 5->1
2
3 1->3
4 5->4
5
6
7 5->7

0
1 5->1
2
3 1->3
4 5->4
5
6 3->6
7 5->7

0 6->0
1 5->1
2 6->2
3 1->3
4 5->4
5
6 3->6
7 5->7



gray:
ineligible

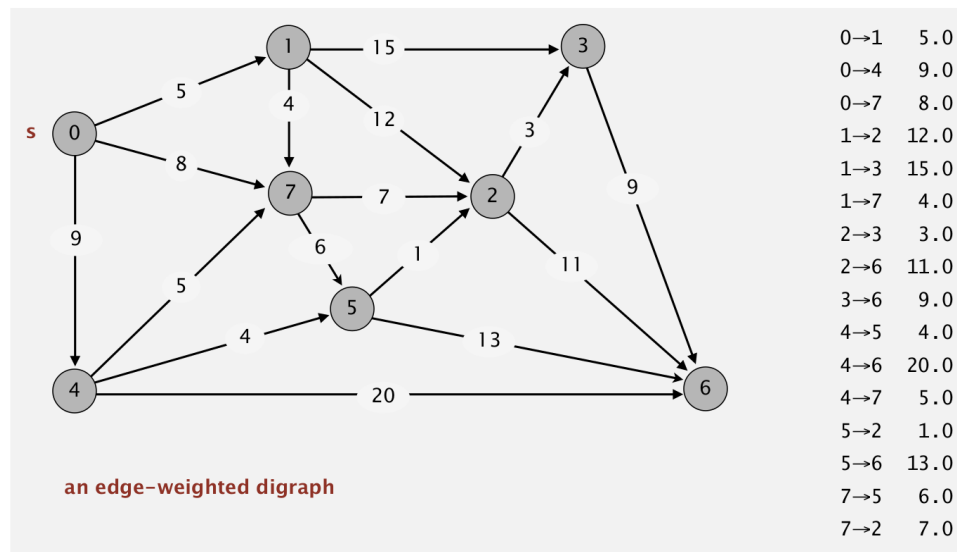


0 4->0
1 5->1
2 6->2
3 1->3
4 5->4
5
6 3->6
7 5->7

0 4->0
1 5->1
2 7->2
3 1->3
4 5->4
5
6 3->6
7 5->7

0 4->0
1 5->1
2 7->2
3 1->3
4 5->4
5
6 3->6
7 5->7

Topological sort per SPT: esercizio



Topological sort: 0 4 1 7 5 2 3 6

Topological sort per SPT: correttezza e complessità

- + Teorema: L'algoritmo di ordinamento topologico trova uno SPT in qualsiasi grafo orientato pesato privo di cicli (pesi negativi sono consentiti) in tempo proporzionale a $E + V$.
- + Dimostrazione:
Ogni arco $e = v \rightarrow w$ viene rilassato una sola volta (da v), lasciando
 $\text{distTo}[w] \leq \text{distTo}[v] + e.\text{weight}()$. La disuguaglianza vale fino alla fine perché $\text{distTo}[w]$ non può aumentare e $\text{distTo}[v]$ non cambierà (essendo i vertici ordinati topologicamente nessun arco che punta a v sarà rilassato dopo che v è rilassato).
Alla fine, vale la condizione di ottimalità.

Topological sort per SPT

```
public class AcyclicSP {
    private double[] distTo;
    private DirectedEdge[] edgeTo;

    public AcyclicSP(EdgeWeightedDigraph G, int s) {
        distTo = new double[G.V()];
        edgeTo = new DirectedEdge[G.V()];

        for (int v = 0; v < G.V(); v++)
            distTo[v] = Double.POSITIVE_INFINITY;
        distTo[s] = 0.0;

        Topological topological = new Topological(G);
        if (!topological.hasOrder())
            throw new IllegalArgumentException("Digraph is not acyclic.");
        for (int v : topological.order()) {
            for (DirectedEdge e : G.adj(v))
                relax(e);
        }

        private void relax(DirectedEdge e) {
            int v = e.from(), w = e.to();
            if (distTo[w] > distTo[v] + e.weight()) {
                distTo[w] = distTo[v] + e.weight();
                edgeTo[w] = e;
            }
        }
    }
}
```

Cammini massimi

- + Dato un grafo orientato pesato e aciclico G , trova il percorso più lungo (o più pesante) dal vertice s al vertice t .
 - + Nega tutti i pesi degli archi;
 - + Trova SP;
 - + Nega i pesi della soluzione.
- + L'algoritmo di ordinamento topologico funziona anche per pesi negativi.

Algoritmo di Bellman-Ford

- + Dato un grafo orientato pesato, uno SPT esiste **se e solo se** non ci sono cicli negativi.
- + Un algoritmo che consente di trovare un SPT in presenza di pesi negativi e di cicli (non negativi):

1. Inizializza $\text{distTo}[s] = 0$ e $\text{distTo}[v] = \infty$ per ogni altro vertice;
2. Ripeti V volte: rilassa tutti gli archi.

```
for ( int i = 0; i < G.V ( ) ; i ++)
```

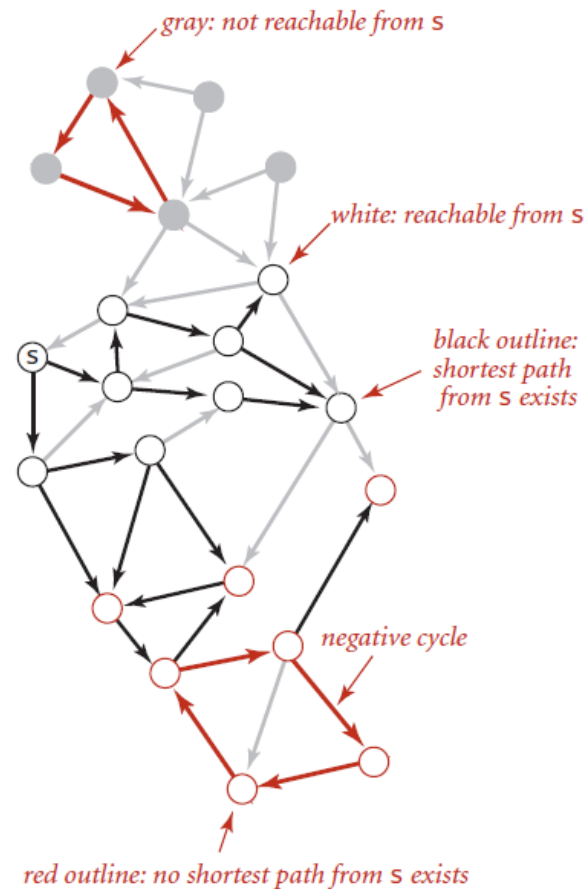
```
    for ( int v = 0; v < G.V ( ) ; v ++)
```

```
        for ( DirectedEdge e : G. adj ( v ) )
```

```
            relax ( e );
```

Cicli negativi

- + Si dimostra che esiste un cammino minimo da s a v in un grafo orientato pesato se e solo se nessun vertice su un cammino da s a v è su un ciclo negativo.



Algoritmo di Bellman-Ford: correttezza e complessità

- + L'algoritmo di Bellman-Ford trova uno SPT in qualsiasi grafo orientato pesato (pesi negativi sono consentiti, cicli non negativi sono consentiti, non sono consentiti cicli negativi raggiungibili da s) in tempo proporzionale a EV , con spazio extra proporzionale a V .
- + Dimostrazione: Dato un vertice t raggiungibile da s , si consideri il cammino minimo (che esiste se non vi sono cicli negativi) $s = v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_k = t$, il quale non può avere più di $V-1$ archi. Dimostriamo per induzione su i che dopo i passi trova il cammino minimo da s a v_i . Per $i=0$ è vero. Assumiamo che l'algoritmo abbia calcolato il cammino minimo $s = v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_i$. Nella i -esima passata rilassiamo tutti i vertici, incluso v_i , e dopo la i -esima passata quindi $\text{distTo}[v_{i+1}] = \text{distTo}[v_i] + e.\text{weight}()$, per $e = v_i \rightarrow v_{i+1}$. Ovvero alla $i+1$ esima passata trovo il cammino minimo fino a v_{i+1} . La complessità in tempo e spazio viene dall'algoritmo.

Algoritmo di Bellmand-Ford con coda

- + Osservazione: Gli unici archi che possono provocare un cambiamento in $\text{distTo}[v]$ sono quelli che escono da vertici il cui valore distTo è stato modificato. Se $\text{distTo}[v]$ non cambia nella passata i -esima, nella passata successiva non c'è bisogno di rilassare gli archi che da esso partono.
- + Mantieni una **coda** dei vertici la cui $\text{distTo}[]$ è cambiata. Si può tenere un array booleano onQ per indicare i vertici che sono in coda (per evitare duplicazioni).
- + Si può controllare la presenza di cicli negativi verificando se si superano i V aggiornamenti di vertice (se c'è un ciclo negativo, l'algoritmo continua ad aggiornare $\text{distTo}[]$ e $\text{edgeTo}[]$ all'infinito). Ovvero c'è un ciclo negativo se la coda non si svuota mai.

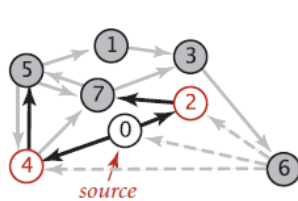
Algoritmo di Bellman-Ford: esempio

tinyEWDn.txt

```
4->5 0.35
5->4 0.35
4->7 0.37
5->7 0.28
7->5 0.28
5->1 0.32
0->4 0.38
0->2 0.26
7->3 0.39
1->3 0.29
2->7 0.34
6->2 -1.20
3->6 0.52
6->0 -1.40
6->4 -1.25
```

queue

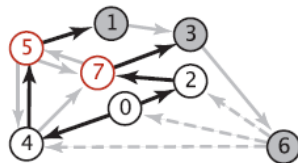
2
4
7
5



edgeTo[] distTo[]

```
0
1
2 0->2 0.26
3
4 0->4 0.38
5 4->5 0.73
6
7 2->7 0.60
```

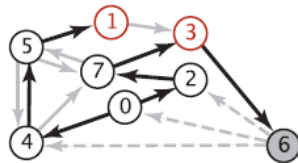
7
5
3
1



edgeTo[] distTo[]

```
0
1 5->1 1.05
2 0->2 0.26
3 7->3 0.99
4 0->4 0.38
5 4->5 0.73
6
7 2->7 0.60
```

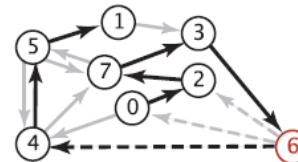
3
1
6



edgeTo[] distTo[]

```
0
1 5->1 1.05
2 0->2 0.26
3 7->3 0.99
4 0->4 0.38
5 4->5 0.73
6 3->6 1.51
7 2->7 0.60
```

6
4

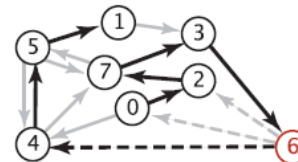


edgeTo[] distTo[]

```
0
1 5->1 1.05
2 0->2 0.26
3 7->3 0.99
4 6->4 0.26
5 4->5 0.73
6 3->6 1.51
7 2->7 0.60
```

no longer eligible!

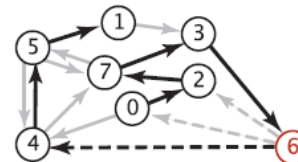
4
5



edgeTo[] distTo[]

```
0
1 5->1 1.05
2 0->2 0.26
3 7->3 0.99
4 6->4 0.26
5 4->5 0.61
6 3->6 1.51
7 2->7 0.60
```

5
1



edgeTo[] distTo[]

```
0
1 5->1 0.93
2 0->2 0.26
3 7->3 0.99
4 6->4 0.26
5 4->5 0.61
6 3->6 1.51
7 2->7 0.60
```

Algoritmo di Bellman-Ford con coda: implementazione

```
public BellmanFordSP(EdgeWeightedDigraph G, int s) {
    distTo = new double[G.V()];
    edgeTo = new DirectedEdge[G.V()];
    onQueue = new boolean[G.V()];
    for (int v = 0; v < G.V(); v++)
        distTo[v] = Double.POSITIVE_INFINITY;
    distTo[s] = 0.0;

    queue = new Queue<Integer>();
    queue.enqueue(s);
    onQueue[s] = true;
    while (!queue.isEmpty() && !hasNegativeCycle()) {
        int v = queue.dequeue();
        onQueue[v] = false;
        relax(G, v);
    }
}

private void relax(EdgeWeightedDigraph G, int v) {
    for (DirectedEdge e : G.adj(v)) {
        int w = e.to();
        if (distTo[w] > distTo[v] + e.weight()) {
            distTo[w] = distTo[v] + e.weight();
            edgeTo[w] = e;
            if (!onQueue[w]) {
                queue.enqueue(w);
                onQueue[w] = true;
            }
        }
    }
}

if (cost++ % G.V() == 0) { // cost = numero di rilassamenti
    findNegativeCycle();
    if (hasNegativeCycle()) return; // found a negative cycle
}
```

Trova un ciclo negativo
se la coda non si svuota
dopo V passate di archi
rilassati

Single source shortest paths: algoritmi a confronto

algoritmo	restrizione	caso peggiore	caso tipico	spazio
topological sort	no cicli	$E + V$	$E + V$	V
Dijkstra (binary heap)	no pesi negativi	$E \log V$	$E \log V$	V
Bellman-Ford	no cicli negativi	EV	EV	V
Bellman-Ford con coda	no cicli negativi	EV	$E + V$	V